

BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Department of Computer Science

COMP333—DATABASE SYSTEMS

Phase 3: Demo Prototype

Prepared by:

Yahya Hasan 1242481 Section 5

Instructor: Yousef Hassouneh

Date: May 24, 2026

Contents

1	Project Overview	2
2	ER Diagram Subset	2
2.1	Entities Implemented	3
3	Implemented Functionalities	4
4	SQL Schema	5
4.1	Table Definitions	5
5	SQL Test Data	7
6	SQL Queries for Each Functionality	7
7	Java Code	9
7.1	JDBC Connection	9
7.2	Login and Sign-Up Logic	10
7.3	Profile Retrieval Logic	12
7.4	Admin Panel Logic	13
7.5	Input / Output Screens (JavaFX)	15

1 Project Overview

The project is **Bawabet Al-Quds Roastery**, an online roastery store system. The demo prototype covers user authentication, customer profiles, and an admin panel with employee and admin management. The application is built with **Java 17 + JavaFX** and connects to a **MySQL** database via JDBC.

2 ER Diagram Subset

The following entities and relationships from the full ER diagram are implemented in this prototype.

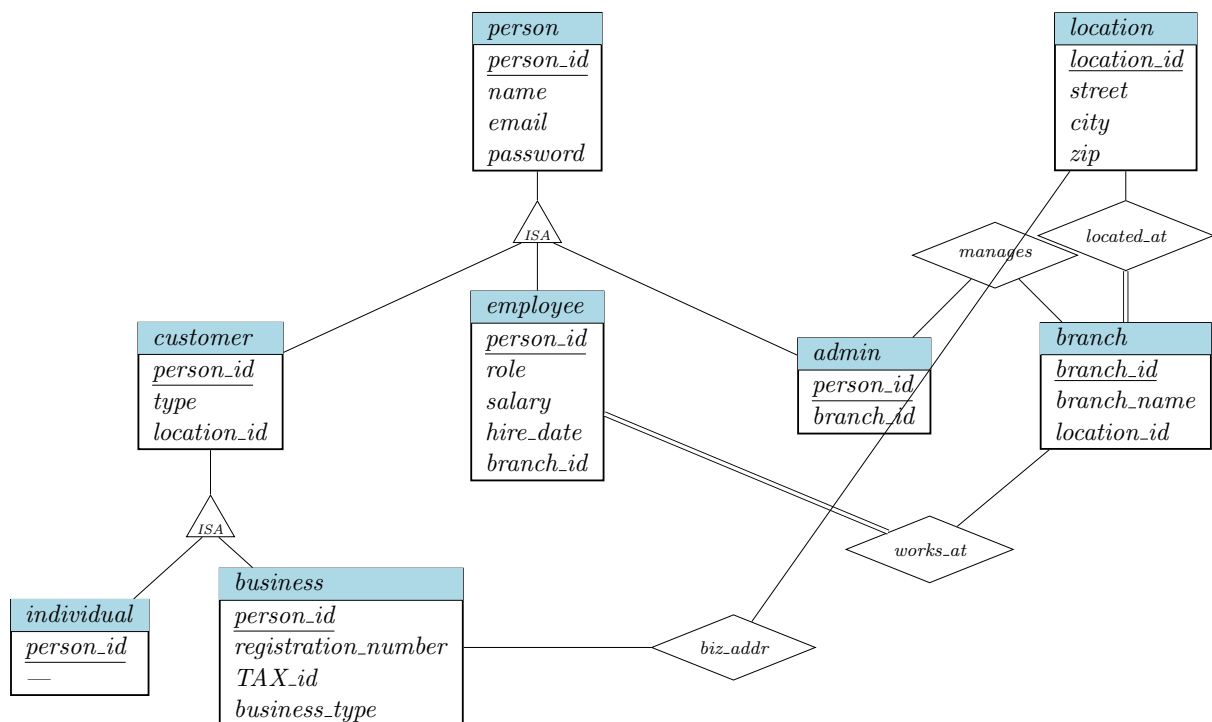


Figure 1: my ERD part

2.1 Entities Implemented

Entity	Role
Person	Supertype for all human actors. Holds name, email, password.
Customer	ISA Person. Discriminated by type (individual / business).
Individual	ISA Customer. Retail buyer subtype.
Business	ISA Customer. Wholesale buyer subtype with registration and tax info.
Employee	ISA Person. Staff assigned to a branch with role and salary.
Admin	ISA Person. Manages a branch.
Location	Address (street, city, zip) shared by Customer and Branch.
Branch	Physical store location, references Location.
Supplier	Provides items to the roastery.

3 Implemented Functionalities

1. User Login

The user provides their name, email, and password on the login screen. The system retrieves the matching row from the `Person` table. If found, the session is opened; otherwise an error label is shown.

2. Register Individual Customer

The user fills in name, email, and password on the sign-up screen (Personal tab). The system inserts one row into `Person`, one row into `Customer` with `type = 'individual'`, and one row into `Individual`. The new user is logged in immediately and redirected to the home page.

3. Register Business Customer

The user fills in name, email, password, address, business type, tax ID, and registration number on the sign-up screen (Business tab). The system inserts rows into `Location`, `Person`, `Customer` (with `type = 'business'`), and `Business`. The new user is logged in immediately.

4. Retrieve Customer Profile

When a logged-in user opens the profile panel, the system retrieves their name, email, and account type by joining `Person` and `Customer` on the stored session `person_id` and displays them on the profile screen.

5. Admin: List All Customers

The admin panel fetches all rows from `Person` joined with `Customer` and displays them in a table view with columns: ID, Name, Email, Type.

6. Admin: List All Employees

The admin panel fetches all rows from `Person` joined with `Employee` and displays them in a table view with columns: ID, Name, Email, Role, Salary, Hire Date, Branch.

7. Admin: Register New Employee

The admin fills in name, email, password, role, salary, hire date, and branch on the Register Employee form. The system inserts into `Person` and then `Employee` using the generated `person_id`.

8. Admin: Register New Admin

The admin fills in name, email, password, and branch on the Register Admin form. The system inserts into `Person` and then `Admin` using the generated `person_id`.

4 SQL Schema

4.1 Table Definitions

```
1 CREATE TABLE Location (  
2     location_id INT AUTO_INCREMENT PRIMARY KEY,  
3     street VARCHAR(100) NOT NULL,  
4     city VARCHAR(45) NOT NULL,  
5     zip VARCHAR(45) NOT NULL  
6 );  
7  
8 CREATE TABLE Branch (  
9     branch_id INT AUTO_INCREMENT PRIMARY KEY,  
10    branch_name VARCHAR(45) NOT NULL,  
11    location_id INT NOT NULL,  
12    FOREIGN KEY (location_id) REFERENCES Location(location_id)  
13 );
```

Listing 1: Location and Branch

```
1 CREATE TABLE Person (  
2     person_id INT AUTO_INCREMENT PRIMARY KEY,  
3     name VARCHAR(45) NOT NULL,  
4     email VARCHAR(45) NOT NULL UNIQUE,  
5     password VARCHAR(100) NOT NULL  
6 );
```

Listing 2: Person supertype

```
1 CREATE TABLE Customer (  
2     person_id INT PRIMARY KEY,  
3     type VARCHAR(45) NOT NULL, -- 'individual' or '  
4     location_id INT,  
5     FOREIGN KEY (person_id) REFERENCES Person(person_id) ON  
6     DELETE CASCADE,  
7     FOREIGN KEY (location_id) REFERENCES Location(location_id)  
8 );  
9  
10 CREATE TABLE Individual (  
11     person_id INT PRIMARY KEY,  
12     FOREIGN KEY (person_id) REFERENCES Customer(person_id) ON  
13     DELETE CASCADE  
14 );  
15  
16 CREATE TABLE Business (  
17     person_id INT PRIMARY KEY,  
18     registration_number VARCHAR(45) NOT NULL,  
19     TAX_id VARCHAR(45) NOT NULL,  
20     business_type VARCHAR(45) NOT NULL,  
21     FOREIGN KEY (person_id) REFERENCES Customer(person_id) ON  
22     DELETE CASCADE  
23 );
```

Listing 3: Customer ISA hierarchy

```

1 CREATE TABLE Employee (
2     person_id INT PRIMARY KEY,
3     role VARCHAR(45) NOT NULL,
4     salary DECIMAL(10, 2) NOT NULL CHECK (salary >= 0),
5     hire_date DATE NOT NULL,
6     branch_id INT NOT NULL,
7     FOREIGN KEY (person_id) REFERENCES Person(person_id) ON
8         DELETE CASCADE,
9     FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
10 );
11
12 CREATE TABLE Admin (
13     person_id INT PRIMARY KEY,
14     branch_id INT NOT NULL,
15     FOREIGN KEY (person_id) REFERENCES Person(person_id) ON
16         DELETE CASCADE,
17     FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
18 );

```

Listing 4: Employee and Admin ISA subtypes

5 SQL Test Data

```
1 -- location and branch needed before any admin or business
  customer
2 INSERT INTO Location (street, city, zip)
3     VALUES ('Main Street 1', 'Ramallah', '00970');
4 INSERT INTO Branch (branch_name, location_id)
5     VALUES ('Main Branch', LAST_INSERT_ID());
6
7 -- admin user seeded directly for the first login
8 INSERT INTO Person (name, email, password)
9     VALUES ('yahya', 'yahya@bawabet.com', 'admin123');
10 INSERT INTO Admin (person_id, branch_id)
11 VALUES (
12     (SELECT person_id FROM Person WHERE name = 'yahya'),
13     (SELECT branch_id FROM Branch WHERE branch_name = 'Main
14     Branch')
15 );
```

Listing 5: Seed data used for testing

Additional test rows are produced at runtime by using the Sign Up screens (individual customers) and the Admin Panel Register forms (employees and admins).

6 SQL Queries for Each Functionality

```
1 SELECT person_id
2 FROM   Person
3 WHERE  email      = '<email>'
4       AND name    = '<name>'
5       AND password = '<password>';
```

Listing 6: Functionality 1 – User Login

```
1 -- step 1: insert the person row
2 INSERT INTO Person (name, email, password)
3     VALUES ('<name>', '<email>', '<password>');
4
5 -- step 2: mark as customer with type individual
6 INSERT INTO Customer (person_id, type)
7     VALUES (<generated_id>, 'individual');
8
9 -- step 3: insert into Individual subtype
10 INSERT INTO Individual (person_id) VALUES (<generated_id>);
```

Listing 7: Functionality 2 – Register individual customer

```
1 -- step 1: insert the address
2 INSERT INTO Location (street, city, zip)
3     VALUES ('<street>', '<city>', '<zip>');
4
5 -- step 2: insert the person row
```



```

6 INSERT INTO Person (name, email, password)
7   VALUES ('<name>', '<email>', '<password>');
8
9 -- step 3: mark as business customer linked to the location
10 INSERT INTO Customer (person_id, type, location_id)
11   VALUES (<person_id>, 'business', <location_id>);
12
13 -- step 4: insert business-specific info
14 INSERT INTO Business (person_id, registration_number, TAX_id,
15   business_type)
16   VALUES (<person_id>, '<reg>', '<tax>', '<kind>');

```

Listing 8: Functionality 3 – Register business customer

```

1 SELECT P.name, P.email, C.type
2 FROM   Person   P
3 LEFT JOIN Customer C ON P.person_id = C.person_id
4 WHERE  P.person_id = <session_person_id>;

```

Listing 9: Functionality 4 – Retrieve customer profile

```

1 SELECT P.person_id, P.name, P.email, C.type
2 FROM   Person   P
3 JOIN   Customer C ON P.person_id = C.person_id;

```

Listing 10: Functionality 5 – List all customers

```

1 SELECT P.person_id, P.name, P.email,
2        E.role, E.salary, E.hire_date, E.branch_id
3 FROM   Person   P
4 JOIN   Employee E ON P.person_id = E.person_id;

```

Listing 11: Functionality 6 – List all employees

```

1 -- step 1: insert person, capture generated id
2 INSERT INTO Person (name, email, password)
3   VALUES ('<name>', '<email>', '<password>');
4
5 -- step 2: insert employee row
6 INSERT INTO Employee (person_id, role, salary, hire_date,
7   branch_id)
8   VALUES (<generated_id>, '<role>', <salary>, '<hire_date>', <
9   branch_id>);

```

Listing 12: Functionality 7 – Register new employee

```

1 -- step 1: insert person, capture generated id
2 INSERT INTO Person (name, email, password)
3   VALUES ('<name>', '<email>', '<password>');
4
5 -- step 2: insert admin row
6 INSERT INTO Admin (person_id, branch_id)
7   VALUES (<generated_id>, <branch_id>);

```

Listing 13: Functionality 8 – Register new admin

7 Java Code

7.1 JDBC Connection

File: src/main/java/app/db_proj/DBConnection.java

```
1 public class DBConnection {
2     private static final String URL =
3         "jdbc:mysql://localhost:3306/bawabet_db"
4         + "?useSSL=false&serverTimezone=UTC&
5           allowPublicKeyRetrieval=true";
6     private static final String USER      = "root";
7     private static final String PASSWORD = "p4ssw0rd";
8
9     private static Connection connection;
10
11     public static Connection getConnection() {
12         try {
13             if (connection == null || connection.isClosed()) {
14                 connection = DriverManager.getConnection(URL,
15                     USER, PASSWORD);
16             }
17         } catch (SQLException e) {
18             throw new RuntimeException(e);
19         }
20     }
21 }
```

Listing 14: Singleton JDBC connection

7.2 Login and Sign-Up Logic

File: src/main/java/app/db_proj/Auth_Logic.java

```
1 public static String login(SystemHandling sys,
2                             String name, String email, String
3                             password) {
4     if (name.isEmpty() || email.isEmpty() || password.isEmpty())
5         return "empty";
6     try {
7         Statement stmt = sys.getConn().createStatement();
8         ResultSet rs = stmt.executeQuery(
9             "SELECT person_id FROM Person WHERE email = '" +
10             email +
11             "' AND name = '" + name + "' AND password = '" +
12             password + "';"
13         );
14         if (rs.next()) {
15             int id = rs.getInt("person_id");
16             sys.setCurrentUser(id, name, email); // stores
17             session + checks admin role
18             return "ok";
19         }
20         return "not_found";
21     } catch (SQLException ex) {
22         System.out.println(ex.getMessage());
23         return "error";
24     }
25 }
```

Listing 15: Login – retrieves person_id and opens session

```
1 public static String signupPersonal(SystemHandling sys,
2                                     String name, String email, String phone, String password)
3     {
4         if (name.isEmpty() || email.isEmpty() || password.isEmpty())
5             return "empty";
6         try {
7             // insert person row and capture the generated id
8             PreparedStatement ps = sys.getConn().prepareStatement(
9                 "INSERT INTO Person (name, email, password) VALUES
10                 (?, ?, ?);",
11                 Statement.RETURN_GENERATED_KEYS
12             );
13             ps.setString(1, name); ps.setString(2, email); ps.
14                 setString(3, password);
15             ps.executeUpdate();
16             ResultSet keys = ps.getGeneratedKeys();
17             int personId = keys.next() ? keys.getInt(1) : -1;
18
19             Statement stmt = sys.getConn().createStatement();
20             stmt.executeUpdate(
```

```

18         "INSERT INTO Customer (person_id, type) VALUES (" +
19             personId + ", 'individual');"");
19     stmt.executeUpdate(
20         "INSERT INTO Individual (person_id) VALUES (" +
21             personId + ");");
21
22     sys.setCurrentUser(personId, name, email);
23     return "ok";
24 } catch (SQLException ex) {
25     if ("23000".equals(ex.getSQLState())) return "duplicate";
26     return "error";
27 }
28 }

```

Listing 16: Sign-up for individual customer

7.3 Profile Retrieval Logic

File: src/main/java/app/db_proj/Profile_Logic.java

```
1 public static CustomerInfo getCustomer(Connection conn, int
   personId) {
2     try {
3         Statement stmt = conn.createStatement();
4         ResultSet rs = stmt.executeQuery(
5             "SELECT P.name, P.email, C.type " +
6             "FROM Person P LEFT JOIN Customer C ON P.person_id =
              C.person_id " +
7             "WHERE P.person_id = " + personId + ";";
8     );
9     if (rs.next()) {
10         return new CustomerInfo(
11             rs.getString("name"),
12             rs.getString("email"),
13             rs.getString("type")
14         );
15     }
16 } catch (SQLException ex) {
17     System.out.println(ex.getMessage());
18 }
19 return null;
20 }
```

Listing 17: Retrieve customer info for the profile screen

```
1 public static boolean isAdmin(Connection conn, int personId) {
2     try {
3         Statement stmt = conn.createStatement();
4         ResultSet rs = stmt.executeQuery(
5             "SELECT 1 FROM Admin WHERE person_id = " + personId +
              ";";
6     );
7     return rs.next();
8 } catch (SQLException ex) {
9     System.out.println(ex.getMessage());
10    return false;
11 }
12 }
```

Listing 18: Check whether the logged-in user is an admin

7.4 Admin Panel Logic

File: src/main/java/app/db_proj/Admin_Logic.java

```
1 public static List<CustomerRow> getCustomers(Connection conn) {
2     List<CustomerRow> list = new ArrayList<>();
3     try {
4         Statement stmt = conn.createStatement();
5         ResultSet rs = stmt.executeQuery(
6             "SELECT P.person_id, P.name, P.email, C.type " +
7             "FROM Person P JOIN Customer C ON P.person_id = C."
8             "person_id;";
9         );
10        while (rs.next()) {
11            list.add(new CustomerRow(
12                rs.getInt("person_id"), rs.getString("name"),
13                rs.getString("email"), rs.getString("type")
14            ));
15        }
16    } catch (SQLException ex) {
17        System.out.println(ex.getMessage());
18    }
19    return list;
20 }
```

Listing 19: Fetch all customers for the admin table view

```
1 public static String registerEmployee(Connection conn,
2     String name, String email, String password,
3     String role, String salary, String hireDate, Integer
4     branchId) {
5     if (name.isEmpty() || email.isEmpty() || password.isEmpty()
6         || role.isEmpty() || salary.isEmpty()
7         || hireDate.isEmpty() || branchId == null)
8         return "empty";
9     try {
10        PreparedStatement ps = conn.prepareStatement(
11            "INSERT INTO Person (name, email, password) VALUES
12            (?, ?, ?);",
13            Statement.RETURN_GENERATED_KEYS
14        );
15        ps.setString(1, name); ps.setString(2, email); ps.
16            setString(3, password);
17        ps.executeUpdate();
18        ResultSet keys = ps.getGeneratedKeys();
19        int personId = keys.next() ? keys.getInt(1) : -1;
20
21        Statement stmt = conn.createStatement();
22        stmt.executeUpdate(
23            "INSERT INTO Employee (person_id, role, salary,
24            hire_date, branch_id) " +
25            "VALUES (" + personId + ", " + role + ", " + salary +
26            ", " + hireDate + ", " + branchId + ");";
27    }
```

```
23     );  
24     return "ok";  
25 } catch (SQLException ex) {  
26     if ("23000".equals(ex.getSQLState())) return "duplicate";  
27     return "error";  
28 }  
29 }
```

Listing 20: Register a new employee from the admin form

7.5 Input / Output Screens (JavaFX)

File: src/main/java/app/db_proj/Auth_UI.java

The login and sign-up screens are built entirely in Java using JavaFX layout nodes. The code below shows the login button handler which connects the UI to the logic layer.

```
1 btn.setOnAction(e -> {
2     warn_empty.setVisible(false);
3     warn_wrong.setVisible(false);
4
5     String result = Auth_Logic.login(sys, name.getText(),
6                                     email.getText(), password.
7                                     getText());
8
9     if (result.equals("empty")) warn_empty.setVisible(true);
10    else if (result.equals("not_found")) warn_wrong.setVisible(
11        true);
12    else if (result.equals("ok")) {
13        // admins go to the admin panel, regular users go to the
14        // home page
15        if (sys.isUserAdmin()) sys.changePage(Page.ADMIN);
16        else sys.changePage(Page.HOME);
17    }
18 });
```

Listing 21: Login button action in Auth_UI

File: src/main/java/app/db_proj/Admin_UI.java

The admin panel displays the customer and employee table views and hosts the register forms. The snippet below shows how the table is populated after the toggle button is pressed.

```
1 employees_btn.setOnAction(e -> {
2     // highlight selected toggle button
3     employees_btn.setBackground(new Background(new BackgroundFill
4         (
5             Color.hsb(48, 1, 0.92, 1), new CornerRadii(10), null)));
6     employees_btn.setTextFill(Color.BLACK);
7     customers_btn.setBackground(Background.EMPTY);
8     customers_btn.setTextFill(Color.hsb(30, 0.12, 0.78, 1));
9
10    // fetch from DB and push into the table view
11    refresh_employees();
12    table_holder.getChildren().setAll(employees_tv);
13 });
14
15 private void refresh_employees() {
16     List<Admin_Logic.EmployeeRow> rows =
17         Admin_Logic.getEmployees(sys.getConn());
18     employees_tv.setItems(FXCollections.observableArrayList(rows)
19 );
20 }
```

Listing 22: Populate employees table view in Admin_UI

File: src/main/java/app/db_proj/ProfileUI.java

The profile panel is a pop-up overlay. On construction it fetches the customer record for the currently signed-in user and displays the data.

```
1 public ProfileUI(SystemHandling sys) {
2     this.sys = sys;
3
4     // pull the current customer info from the database when the
5     profile opens
6     if (sys.getCurrentUserId() != null) {
7         info = Profile_Logic.getCustomer(sys.getConn(), sys.
8             getCurrentUserId());
9     }
10    // ...
11 }
12
13 // inside Base() -- display the retrieved data
14 String displayName = info != null && info.name != null ? info.
15     name : "Name";
16 String displayEmail = info != null && info.email != null ? info.
17     email : "";
18 String displayType = info != null && info.type != null ? info.
19     type : "";
20
21 Label name = new Labels(displayName, ...).getLabel();
22 Label emailLbl = new Labels(displayEmail, ...).getLabel();
23 Label typeLbl = new Labels(displayType, ...).getLabel();
```

Listing 23: Profile data retrieval in ProfileUI constructor